FinSight

Dados financeiros em perspectiva

Engenharia de Software e Arquitetura de Sistemas

Design de software e UML aplicados ao produto

Identificação	Entrega 2
Grupo	6 - DevLeaders
Integrantes	Gustavo Pires; Lúcio Vecchio; Luiz Henrique Zaim da Cruz; Eric Aloise
Orientação	Luis Pires
Data de referência	08/11/2026
Versão do código	a32b70b1c53c
Repositório	github.com/2026-2-NCC4/Projeto6

1. Arquitetura implementada

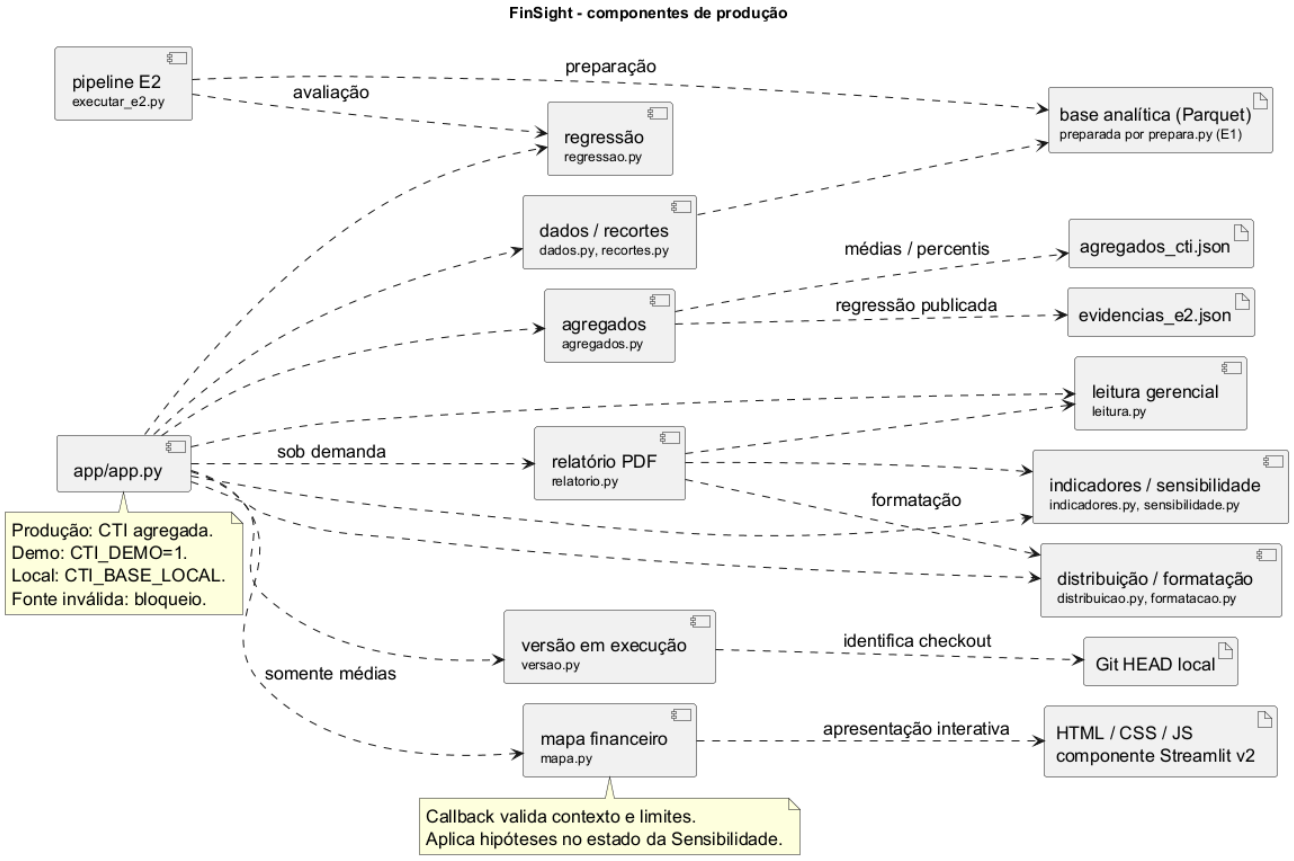
Arquitetura modular em Python: a interface Streamlit coordena serviços de domínio; preparação e validação isolam os dados; cálculos financeiros e regressão são reutilizáveis por CLI, notebook e painel. O sistema não exige banco nem API próprios para o volume atual.

Entradas: CSV CTI autorizado ou Parquet preparado; gerador fictício independente para demonstração; filtros de ano e repetição; hipóteses de desconto, custo, volume e meta. Processamento: validação, recorte, médias anuais, KPIs, simulação e ajuste por trajetórias. Saídas: tabelas, gráficos, indicadores com estado, PDF gerencial do recorte, PDFs acadêmicos, evidências e CSV de indicadores agregados reais.

Decisões: reaproveitar prepara.py evita divergência com E1; funções puras separam regra e apresentação; Parquet reduz custo de leitura; cache evita repetir carga e ajuste sem alteração de dados. O modo público não inclui upload ou exportação de dados privados. O mapa Streamlit v2 recebe somente médias de contas autorizadas, deriva margem e renderiza HTML/CSS/JS próprios. O callback valida recorte e limites antes de aplicar desconto e custo ao estado da Sensibilidade; o modelo permanece no Python.

Alternativa de banco/API aumentaria manutenção sem requisito atual de concorrência de escrita. A arquitetura admite substituição futura do carregador sem duplicar fórmulas.

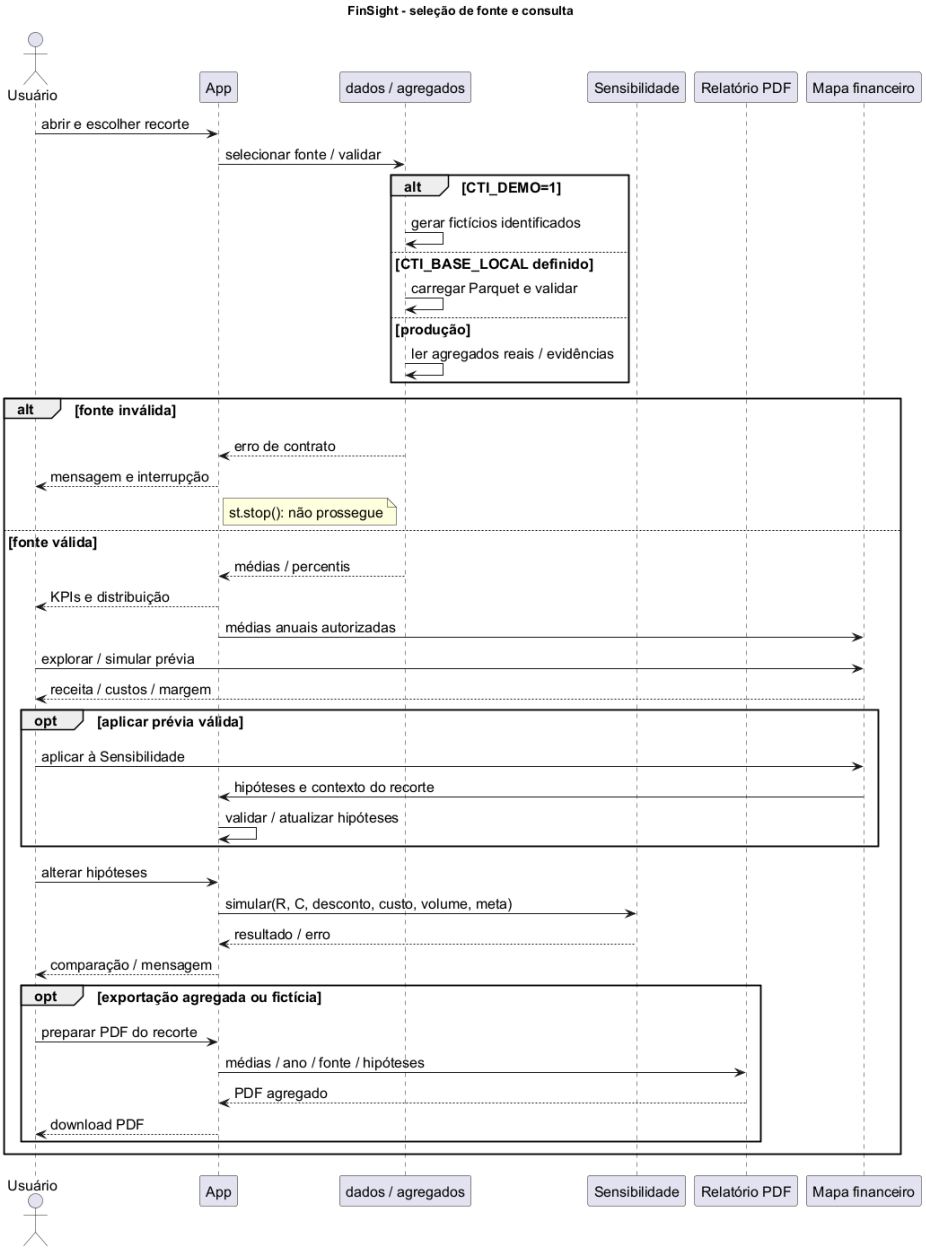
2. UML de componentes



O consumidor aponta para o provedor por dependência tracejada. app utiliza dados/recortes, agregados, indicadores, sensibilidade, distribuição, formatação, regressão, versão, leitura gerencial, mapa financeiro e relatório PDF. O relatório reutiliza regras financeiras e leitura gerencial. executar_e2 prepara e avalia; a versão pública lê agregados_cti.json e evidencias_e2.json. A base completa é local; demonstração é uma opção explícita.

As fontes editáveis dos diagramas (arquivos .puml) ficam na mesma pasta deste PDF; as figuras são renderizadas dessas fontes e incorporadas pelo gerador do documento. Dependências representam consumo de módulos e artefatos; não são ordem de execução.

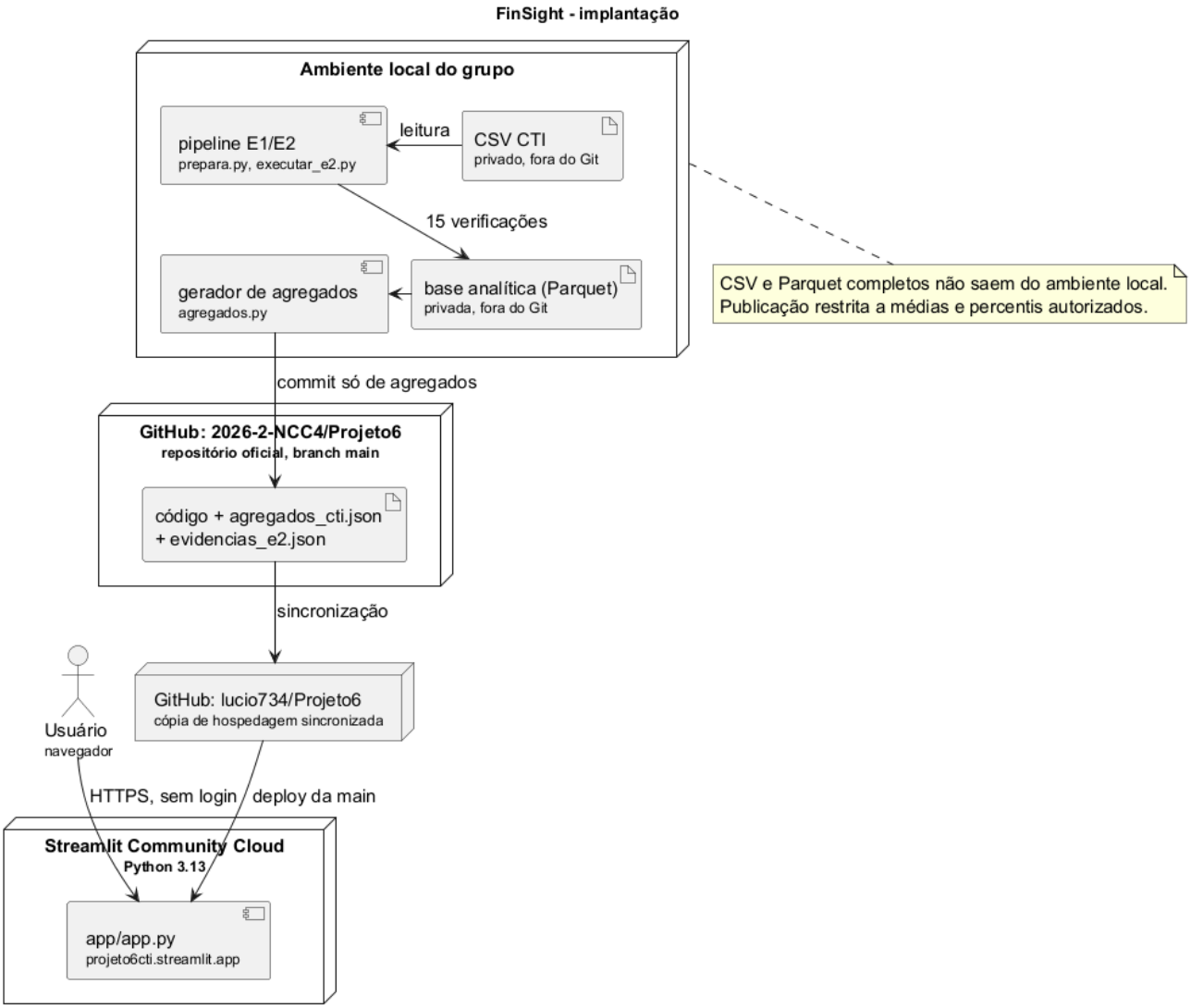
3. UML de sequência



Participantes: usuário, app, dados/agregados, mapa financeiro, sensibilidade e relatório PDF. Linhas de vida tracejadas, mensagens e retornos; fragmento alt distingue entrada inválida e válida. Resultado com falha é apresentado como mensagem, nunca substituído silenciosamente por zero.

O processamento da simulação recebe cópia dos valores de referência. Restaurar hipóteses e a troca de ano/população reinicializam a meta na margem do recorte; não escrevem na base. No modo agregado, a regressão lê evidências já calculadas; no modo local, é iniciada por comando explícito. Seu recorte 1-11 é informado separadamente.

4. UML de implantação



O diagrama mostra onde cada parte roda. O CSV da CTI e a base analítica ficam no ambiente local do grupo, fora do Git. O pipeline gera os agregados autorizados, versionados no repositório oficial junto com o código. Uma cópia de hospedagem sincronizada alimenta o Streamlit Community Cloud, que entrega o painel ao navegador por HTTPS, sem login.

A cópia de hospedagem existe porque a publicação no Streamlit exige permissão de administrador no repositório, que a conta do grupo não tem no repositório oficial. A sincronização mantém os mesmos commits, e o painel exibe o commit em execução para conferência.

5. Rastreabilidade e qualidade

Requisitos FECAP	Implementação / evidência
RF01-RF05: fontes e preparação	prepara.py E1 + dados.py; hash preservado, 15 verificações e cobertura por cenário
RF06-RF07: análise e regressão	E1 descritiva preservada; regressao.py + notebook E2 com 6/6 células
RF08 / RF11: KPIs e cenários	indicadores.py e sensibilidade.py; 216 KPIs + 24 identidades conferidos
RF09-RF10 / RF13: relatório e painel	Quatro PDFs; cinco cartões, evolução, tabelas, filtros e notas
RF12: exportação autorizada	CSV e PDF do recorte agregado disponíveis; base completa e linhas individuais não exportadas
RF14: execução e publicação	Painel publicado no Streamlit Community Cloud: https://projeto6cti.streamlit.app/. O commit em execução aparece na aba Metodologia.
Confiabilidade / manutenção	56 testes aprovados, incluindo entradas inválidas, recortes, fórmulas, modelo, interface, versão e teste negativo real
Reprodução / segurança	Dependências específicas E2; base completa ignorada pelo Git; produção CTI agregada como padrão

A suíte utiliza casos independentes com valores conhecidos e comparação com planilha E1; não verifica apenas que o código executa. Dados essenciais ausentes, chave duplicada, cobertura incompleta e simulação inválida bloqueiam os respectivos resultados.

6. Resultados e reflexão

Esperado: integrar as disciplinas em um único painel, preservar a base original, manter a consistência dos números e publicar a solução. Obtido: painel publicado em nuvem com cinco indicadores, filtros, distribuição entre cenários, financeiro, sensibilidade, regressão e exportações em CSV e PDF; preparação real e conferência financeira sem divergências; notebook executado; documentos gerados das mesmas evidências.

Divergência relevante: a regressão apresenta colinearidade alta e forte dependência temporal. A solução adotou divisão por trajetória, erros-padrão agrupados e interpretação associativa, e a limitação foi documentada em vez de buscar um indicador de ajuste mais alto.

Mudanças durante a entrega: a publicação exigiu permissão de administrador que a conta do grupo não tem no repositório oficial; a solução foi uma cópia de hospedagem sincronizada, sem alterar as permissões. O painel público passou de dados fictícios para agregados autorizados da CTI, com contrato validado e bloqueio quando falta uma fonte válida. A revisão cruzada encontrou dois defeitos de interação (meta presa ao primeiro ano e aviso de estado ao restaurar hipóteses), corrigidos com teste de regressão. Os UML passaram a ser renderizados das fontes versionadas, e o painel passou a exibir o commit em execução.

A preparação da E2 reutiliza a da E1 sem sobrescrever seus relatórios, e o código específico de plataforma foi separado do ambiente Windows usado na primeira entrega.

Melhorias futuras: esclarecer convenções CTI, testar estabilidade do modelo em outras partições, avaliar modelos de dependência temporal e acrescentar autenticação somente se houver publicação autorizada de dados restritos. O protótipo não sustenta decisão econômica real sem validação do domínio.

Referências: enunciado FINAL FECAP, seções 4 a 6 e 11; artefatos da E1; módulos e testes da E2 no repositório.